

ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

DnD Above

2025.

Szakma:

SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ
5 0613 12 03

Készítették:

Gulyás András Zalán
Tóth Kristóf
Török Máté József

Tartalomjegyzék

I.	Bevezetés	5
II.	Témaválasztás indoklása	6
III.	Fejlesztői dokumentáció, technikai leírás.....	7
	Specifikáció	7
	Az alkalmazott fejlesztői eszközök	8
	Adatmodell leírása.....	9
	Felhasználóhoz kapcsolódó táblák	9
	Races.....	10
	Classes	10
	Subclasses	11
	Equipments	11
	Features.....	12
	Spells	13
	Algoritmusok.....	14
	Backend	14
	Frontend	16
	Tesztelési dokumentáció.....	24
	Teszteset #1 – Helyes Felhasználó létrehozása	24
	Teszteset #2 – Hibás Felhasználó létrehozása	24
	Teszteset #3 – Helyes karakterkészítés	25
	Teszteset #4 – Hibás karakterkészítés	26
	Teszteset #5 – Elkészített karakter mentésének kihagyása	27
IV.	Felhasználói dokumentáció	28
	A program általános specifikációja, célja.....	28
	Rendszerkövetelmények.....	29

Hardverkövetelmények:.....	29
Szoftverkövetelmények:	29
A program telepítése.....	30
1. Fájlok előkészítése (1).....	30
2. Fájlok előkészítése (2).....	30
2. Tűzfal- vagy vírusirtó figyelmeztetés	30
3. A program indítása	30
4. A program bezárása.....	31
5. Fontos megjegyzés	31
A program használatának a részletes leírása	32
A program használatának megkezdése.....	32
Karakterkészítés lépésről lépésre.....	32
Egyéb funkciók leírása	35
V. Összegzés	36
Általános Összegzés	36
További fejlesztési lehetőségek	36
Nem megvalósított funkciók	36
VI. Ábrajegyzék.....	38
VII. Irodalomjegyzék, hivatkozásjegyzék	39

Nyilatkozat

Alulírott Név büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 18.

.....*Tóth Kristóf*.....

Tóth Kristóf

Nyilatkozat

Alulírott Név büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 18.

.....*Török Máté József*.....

Török Máté József

Nyilatkozat

Alulírott Név büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2025. április 18.

.....*Gulyás András Zalán*.....

Gulyás András Zalán

I. Bevezetés

A Dungeons & Dragons, továbbiakban DnD egy a 90'-es évek óta futó asztali-szerepjáték, amelynek a legújabb, 5. kiadásához kapcsolódik a DnD Above Projekt.

A DnD egy fantasy asztali-szerepjáték, amelyben a játékosok általuk készített és kitalált karakterek bőrébe bújva fedeznek fel egy fantasy történetet, amelyet egy Dungeon Master-el, vagy játékmesterrel közösen élnek át.

A játékosok karaktereinek vannak különböző képességei és tulajdonságai, amelyeket az adott karakter 'Character Sheet'-én tartják számon.

Ezen sheet megírása a karakter elkészítésekor új játékosok számára a játékmester segítségével elég bonyolult feladat lehet, mivel nagyon sok mindent kell kiválasztani és feljegyezni, és ezek sok különböző kiegészítőből és könyvből jöhetnek.

Ilyenek például teljesen új Class-ok, mint az Artificer a 'Tasha's Cauldron of Everything', amely teljesen új lehetőségeket biztosít mágikus tárgyak szintjén a játékosoknak, akik ezt választják. De nem csak hivatalosan a Wizards of the Coast által kiadott tartalmak használhatók, hanem külön felek által készített és forgalmazott kiegészítők, például a MonkeyDM által kiadott 'Steinhardt's Guide to the Eldritch Hunt' amely a Jaeger Class-t mutatja be. Ez a Class több új mechanikát implementál, például a Momentum, Finisher-ek vagy Focus Art-ok.

De rengeteg új Subclass, vagy al-Class is megtalálható ezekben a kiegészítőkben, mint a 'Xanathar's Guide to Everything', amelyben megjelent a Warlock Class 'Hexblade' Subclass-e, amely egy az árnyékvilágból származó kardot használ, mint Patrónusa.

Ezek mellett új fajokat is kreáltak, például a fentebb említett 'Steinhardt's Guide to the Eldritch Hunt'-ban a Manikin faj, amik életre keltett bábuk, amelyek varázslat által működnek, és több úgy nevezett alfajjal is rendelkeznek.

A DnD-hez készültek még varázslatok is, például a szintén a 'Steinhardt's Guide to the Eldritch Hunt'-ban található 'Gravity Well' ami egy nagyon erős és hasznos 1-es szintű varázslat.

Mindezek miatt, és a játék alapvető bonyolultsága és viszonylag nehezen tanulhatósága miatt, sokaknak, főleg kezdőknek kihívást jelent a játék sok információját számontartani a sheet-en.

II. Témaválasztás indoklása

Az előbbieken kifejtettek és személyes tapasztalatok miatt döntöttünk úgy, hogy elkészítjük a 'DnD Above' projektet, hogy segítsünk játékosoknak elkészíteni és szamon tartani a character sheet-eiket.

Ez egy nagyon hasznos dolog lehet sokaknak, főleg mivel a 2023-ban megjelent 'Dungeons and Dragons: Honor Among Thieves' népszerűbbé tette a DnD-t továbbá sok új játékosnak mutatta be 'A hobbit', illetve a 'Stranger Things' sorozatban is többször van megemlítve a játék és annak szörnyei, például a Demogorgon.

Kutatások szintjén leginkább személyes tapasztalatainkat használjuk fel, valamint az évek alatt szerzett tudásunkat a játékról, annak rendszereiről és a hozzá kiadott tartalmakról.

Ez mellet több másik játékost is megkérdeztünk hogy mennyire találják bonyolultnak a karakter kreációs folyamatot, illetve a sheet-ek kezelését valamilyen erre specifikusan kifejlesztett program nélkül.

Eddigi játékaink során mi leginkább egy .pdf character sheet-et használtunk, amit Adobe Acrobat Reader-el szerkesztettünk, de ez sokaknak kényelmetlen, bonyolult vagy egyszerűen csak elavult módszer.

Emellett, az információkat a játék különböző tartalmairól sok munka és idő lehet a sok forrásból összegyűjteni, hiába van néhány weboldal, ahol az információ egy része megtalálható, például a Wikidot.com.

Ezen eszközökre alapozva és ezek működését és használhatóságát tovább fejlesztve tervezzük megalkotni a DnD Above-t, amely magában fog foglalni egy Character Sheet készítőt, illetve szerkesztőt, és egy adatbázist, ami tartalmazza a különböző, DnD 5E SRD-ben megtalálható Class-okat, Subclass-eket, fajokat, tárgyakat és varázslatokat.

III. Fejlesztői dokumentáció, technikai leírás

Specifikáció

Egy webalkalmazás amely DnD character sheet-k elkészítésére és tárolására használható, pontosabb felsorolás a funkcióiról:

- Elkészített character sheet-k tárolása
- Karakter készítés az adatbázisban tárolt források használatával
- A sheet-en található kiszámítandó értékek kiszámolása a sheet elkészítésekor (pl.: életerő, különböző bónuszok, szint és alap karakter attribútumok alapján változó értékek)
- Képességek elrendezése és kiírása
- Varázslatok tárolása, kiírása, rendezése (pl.: szint alapján)
- Készségek (értsd: Skill-ek) tárolása, bónusz értékek kiszámolása
- Karakter háttér
- Karakter által birtokolt tárgyak és pénz tárolása
- Felhasználókezelés
- Webes felület
- Online elérhető felület és funkciók
- Effektív és hatékony character sheet dizájn, és szerkesztési, illetve készítési felhasználói felület

Az alkalmazott fejlesztői eszközök

- Programozási Nyelvek
 - C#
 - JavaScript
 - HTML
 - CSS
- Fejlesztői környezet
 - Microsoft Visual Studio 2022
 - Microsoft Visual Studio Code
 - .NET
- Adatbázis-kezelő rendszer
 - SQL
 - DB Browser
- Adatbázismotor
 - SQLite
- Szövegszerkesztő program
 - Microsoft Word
 - Jegyzetömb
- Képszerkesztő program
 - Gimp
 - Adobe Photoshop
 - Microsoft Paint
- Keretrendszerek
 - React
 - Vite

Adatmodell leírása

Az adatbázisban táblát használtunk, ezek a következők:

- Users
- Saves
- Races
- Classes
- Subclasses
- Features
- Spells
- Equipments
- RefreshTokens

Az adatbázisban logikai értékek helyet integerként, 0-ként (azaz hamis) és 1-esként (azaz igaz), tároljuk. Adott táblák nem minden oszlopa van elmagyarázva mert magától értetődőnek találtuk őket (pl.: Name).

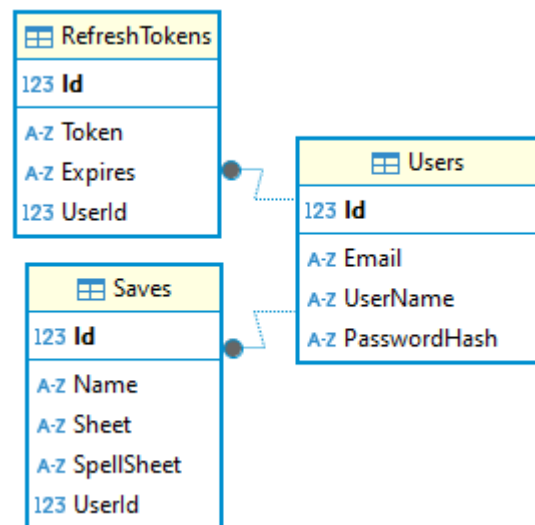
Felhasználóhoz kapcsolódó táblák

A Saves tábla 'UserId' oszlopa referál a Users tábla 'Id' oszlopára (amely a Users tábla elsődleges kulcsa), ahogy a képen is látható.

A 'Sheet' oszlop eltárolja a character sheet teljes dizájnját és adatait, kivéve a varázslatokat, amit később a frontend le tud kérni és kiíratni.

A SpellSheet oszlop tárolja a varázslatokat, ha vannak, erre azért szükség mert nem fért el egy oszlopba mert extrém esetben túl sok adat volt.

A Users tábla 'UserName' oszlopa a felhasználónevet tárolja el, a 'PasswordHash' oszlop a felhasználó jelszavát titkosítva, az 'Email' oszlop a felhasználó által megadott e-mail címet.



1. ábra - Felhasználóhoz kapcsolódó táblák

Races

A 'CreatureType' a faj típusát tartalmazza például Humanoid.

A 'Size' oszlop a faj átlagos méretbeli besorolását tartalmazza, amely általában Medium vagy Small, azaz kicsi vagy közepes lehet, amelyet néhány játék belli hatás használ.

A 'Speed' azt a távolságot írja le, amit az adott faj átlagos tagja a harc egy köre alatt (értsd: 6 másodperc) meg tud tenni.

Races	
123	Id
A-Z	Name
A-Z	Description
A-Z	CreatureType
A-Z	Age
A-Z	Size
123	Speed
A-Z	Languages

2. ábra - Races tábla

Classes

A 'HitDice' oszlopban a Class által használt, a maximum életerő meghatározására dobott kocka van tárolva ('d', majd a 'dobókocka oldalainak száma' formátumban).

A 'StartingGold' oszlopban a Class által karakter készítéskor kapott arany mennyisége van eltárolva.

A 'SpellCaster' és 'HalfCaster' oszlopokban, amelyek egy 0 vagy 1 értékű integerek, megadjuk, hogy az adott Class rendelkezik-e a spellcasting feature-el, azaz tud-e varázslatokat használni, és hogy milyen módszerrel számítható ki, hogy mely szinteken kap hozzáférést magasabb szintű varázslatokhoz.

A 'SavingThrows', 'WeaponProfs', 'ArmorProfs' és 'ToolProfs' arra vannak használva, hogy eltároljuk hogy az adott Class milyen szakértermekkel rendelkezik fegyverekhez, páncélokhoz, eszközökhöz és adott, általában hirtelen, hatások elleni védelemhez (pl.: egy robbanás, egy csapda).

Classes	
123	Id
A-Z	Name
A-Z	Description
A-Z	HitDice
123	StartingGold
123	SpellCaster
123	HalfCaster
A-Z	ArmorProf
A-Z	WeaponProf
A-Z	ToolProf
A-Z	SavingThrows

3. ábra - Classes tábla

Subclasses

Az 'OriginClassId' idegen kulcs oszlop referál a Classes tábla 'Id' oszlopára, ezzel megmutatva melyik Class-hoz tartozik a Subclass, és ez alapján, hogy mely Class-ban kell adott szintet elérni, hogy a karakter ezzel a Subclass-el rendelkezzen.

SubClasses	
123	Id
A-Z Name	
123	OriginClassId
A-Z Description	

4. ábra - SubClasses tábla

Equipments

A 'ProfReq' oszlop eltárolja hogy az adott tárgy használatához a karakternek szüksége van-e ahhoz a tárgyhoz, vagy tárgyfajtaához specifikus képzettségre (pl.: egy hosszúkard effektív használatához szükséges ilyen képzettség).

Az 'EquipmentType' oszlop az adott tárgy típusát mutatja (csak fegyverek, páncélok és pajzsok esetében), amelyet az előbb említett 'ProfReq' oszloppal együtt használunk arra, hogy megállapítsuk egy tárgyhoz milyen hozzáértés vagy képzettség szükséges, és hogy az adott karakter rendelkezik-e ilyen képzettséggel.

A 'DamageDie' és 'DamageType' oszlopok az adott tárgy (legtöbb esetben fegyver) által kiosztott sebzéshez használt kocka oldalainak számát tárolja el, a fentebb leírt formátumban és a kiosztott sebzésnek a típusa (például egy lándzsa szúrással támad ezért 'piercing' sebzést okoz, vagy egy tűzgolyó égési sérülést okoz).

A 'Properties' oszlop eltárolja a tárgy tulajdonságait (pl.: fegyvereknél, hogy egy- vagy kétkézes).

Az 'Attunement' oszlop, amely egy 0 vagy 1 értékű integer, csak mágikus tárgyak esetében lényeges, és azt tárolja, hogy az adott tárgy használatához a karakternek speciális kapcsolatot kell-e létrehoznia a tárggyal (legtöbbször egy rituálé keretében).

Az 'AC' oszlop, amely egy 0 és 20 közötti értékű integer, a tárgy által biztosított védelmi értéket tárolja el. Ez az oszlop csak páncéloknál és pajzsoknál lényeges.

A 'Consumable' oszlop, amely egy 0 vagy 1 értékű integer, azt tárolja el, hogy a tárgy fogyóeszköz-e (pl.: egy bájital vagy élelem).

A 'Rarity' oszlop a tárgy ritkaságát tárolja el. Ez lehet Common, Uncommon, Rare, Very Rare, Legendary vagy Artifact ritkaság, a legkevésbé ritkától a legritkábbig.

Equipments	
123	Id
A-Z Name	
A-Z Rarity	
123	ProfReq
A-Z EquipmentType	
A-Z DamageDie	
A-Z DamageType	
123	AC
A-Z Properties	
123	Consumable
123	Price
A-Z Description	
123	Attunement

5. ábra - Equipments tábla

Features

A 'LevelReq' oszlop tárolja azt, hogy az adott képesség birtoklásához vagy megszerzéséhez hányas szintűnek kell minimum lennie a karakternek. A fentebb leírtak alapján az 'OriginClassId', 'OriginSubClassId', 'OriginEquipmentId' és 'OriginRaceId' oszlopok pedig (Class esetében) ezzel együtt megmutatják melyik Class-ban kell ezt a szint minimumot a karakternek elérnie.

A 'SavingThrows', 'WeaponProf', 'ArmorProf', 'SkillProf' és 'ToolProfs' oszlopok itt is a fentiekben leírtak alapján funkcionálnak, azzal a kivétellel, hogy egy képesség nem minden esetben rendelkezik mindegyikkel, illetve bizonyos esetekben nem rendelkezik egyikkel sem.

Features	
123	Id
A-Z	Name
123	LevelReq
A-Z	Description
A-Z	ArmorProf
A-Z	WeaponProf
A-Z	ToolProf
A-Z	SavingThrows
A-Z	SkillProf
123	OriginClassId
123	OriginSubClassId
123	OriginRaceId
123	OriginEquipmentId

6. ábra - Features tábla

Spells

A 'School' oszlop tárolja el azt, hogy a varázslat mely mágikus „iskolába” sorolható be (pl.: Abjuration, Conjuraton, Necromancy).

A 'Concentration' oszlop, amely egy 0 vagy 1 értékű integer, azt tárolja, hogy az adott varázslatra a használójának koncentrálnia kell-e. Ez azért lényeges, mert ebben az esetben a varázslat hatása megtörhető, ha a használó koncentrációját megtörik, sebzésen vagy más hatásokon keresztül.

A 'Range' oszlop a varázslat effektív távolságát tárolja, amely általában lábban van megadva, de lehet speciális esetekben „Self”, vagyis a használó saját magán tudja csak használni, vagy „Touch”, azaz érintés, amely esetben a használó csak olyan célponton használhatja, akit megérint (a játékban általában ez maximum 5 láb távolságot jelent).

A 'Ritual' oszlop, amely egy 0 vagy 1 értékű integer, azt tárolja el, hogy az adott varázslat használható-e 10 perces rituáléként, amely esetben a varázslat nem kerül mágikus erőforrásokba.

A 'CastingTime' oszlop eltárolja, hogy az adott varázslatot mennyi időbe tart használni. Ez legtöbb esetben a harcok alatt használt kifejezések valamelyike (Action, Bonus Action vagy Reaction) vagy egy időtartam.

A 'Component' és a 'ComponentPrice' oszlopok megmutatják a varázslat használatához milyen komponensekre van szükség, és azok árát, ha van nekik.

A 'LearnedBy' azon Classok azonosítóját tárolja mely megtudja tanulni az adott varázslatot. Az 'OriginClassId', 'OriginSubClassId', 'OriginEquipmentId' és 'OriginRaceId' oszlopok mind idegen kulcsok, melyek a nevükben lévő táblában található 'Id' azonosító értékét veszik fel, ezzel eltárolva, honnan van az adott karakternek ez a Feature-e vagy képessége.

Spells	
123	Id
A-Z	School
123	Concentration
A-Z	Name
123	Level
A-Z	Description
A-Z	Range
A-Z	Duration
123	Ritual
A-Z	CastingTime
A-Z	Component
123	ComponentPrice
A-Z	LearnedBy
123	OriginEquipmentId

7. ábra - Spells tábla

Algoritmusok

Backend

Dto-k

Dto-kat készítettünk a Users és a Saves táblákhoz, ilyen például a RegisterDto mely csak regisztráció alatt frontendtől kapott változókat tárolja például a felhasználónév.

```
namespace Above_backend.Models.DTOS
{
    2 references
    public class RegisterDto
    {
        1 reference
        public string UserName { get; set; }
        2 references
        public string Email { get; set; }
        1 reference
        public string Password { get; set; }
    }
}
```

8. ábra - RegisterDto modell

Továbbá ilyen még a SavesBaseDisplayDto melyben a 'Saves' táblának csak két lényeges elemét adja vissza a frontendnek, mely majd a nevet fogja megjeleníteni és az Id-val fogják majd lekérni a kiválasztott sorát a 'Saves' táblának melyben majd

```
public class SavesBaseDisplayDto
{
    1 reference
    public int Id { get; set; }
    1 reference
    public string Name { get; set; }
}
```

9. ábra - SaveBaseDisplayDTO model

szereppel egyéb adat is, a Sheet és SpellSheet nevű adat tag, ezért neveztük el BaseDisplayDto-nak valamint van egy SheetDisplayDto is.

További modellek

A RefreshToken tábla a kiadott tokeneket tárolja ezzel ellenőrizve hogy egy adott felhasználónak be kell-e újra jelentkeznie mert lejárt a tokenje ezzel növelve a biztonságot.

```
public class RefreshToken
{
    0 references
    public int Id { get; set; }
    5 references
    public string Token { get; set; }
    2 references
    public DateTime Expires { get; set; }
    1 reference
    public int UserId { get; set; }
    3 references
    public User User { get; set; }
}
```

10. ábra - RefreshToken model

A FeaturesToFeaturesConnection tábla, továbbiakban kapcsolótábla, egy specifikus problémánkat oldja meg miszerint a program megfelelő működéséhez nekünk szükségünk van arra, hogy tudjunk Feature-öket egymásból eredeztetni.

```
namespace Above_backend.Models
{
    7 references
    public class FeaturesToFeaturesConnection
    {
        3 references
        public int Id { get; set; }
        2 references
        public int OriginFeatureId { get; set; }
        2 references
        public int SubFeatureId { get; set; }
    }
}
```

11. ábra - FeaturesToFeaturesConnection model

Helpers

Helpers mappát hoztunk létre melyekbe Mapping fájlokat készítettünk a Saves és a Users táblák különböző dto-ihoz.

Ilyen például a MappingSaves fájl melyben két fajta dto, a SavesBaseDisplayDTO-vá és SavesSheetDisplayDTO-vá alakítunk Saves típusú elemeket, valamint van egy mely SavesCreateDTO-t alakít Saves formátummá.

Másik fájl a Helpers mappában a MappingUsers mely egy UserDisplayDTO-vá alakít User típusú elemet, valamint RegisterDTO-t alakít User-é.

```
public class MappingSaves
{
    1 reference
    public static SavesBaseDisplayDTO SavesToSavesBaseDisplayDTO(Saves saves)
    {
        return new SavesBaseDisplayDTO
        {
            Id = saves.Id,
            Name = saves.Name,
        };
    }

    1 reference
    public static Saves SavesCreateDTOToSaves(SavesCreateDTO savescreatedto)
    {
        return new Saves
        {
            Name = savescreatedto.Name,
            Sheet = savescreatedto.Sheet,
            UserId = savescreatedto.UserId,
        };
    }

    1 reference
    public static SavesSheetDisplayDTO SavesToSavesSheetDisplayDTO(Saves saves)
    {
        return new SavesSheetDisplayDTO
        {
            Id = saves.Id,
            Sheet = saves.Sheet,
        };
    }
}
```

12. ábra - MappingSaves kód részlet

```
public class MappingUsers
{
    1 reference
    public static UsersDisplayDTO UserToUserDisplayDTO(User user)
    {
        return new UsersDisplayDTO
        {
            Id = user.Id,
            UserName = user.UserName,
            Email = user.Email,
            CreatedAt = System.DateTime.Now,
            UpdatedAt = System.DateTime.Now,
        };
    }

    1 reference
    public static User RegisterUserToUser(RegisterDTO userregisterdto)
    {
        return new User
        {
            CreatedAt = System.DateTime.Now,
            UpdatedAt = System.DateTime.Now,
        };
    }
}
```

13. ábra - MappingUsers kód részlet

```
        CreatedAt = System.DateTime.Now,
        UpdatedAt = System.DateTime.Now,
    };
}
```

Frontend

Adatátvitel Komponensekről

Több Context-et használunk, hogy a különböző komponensekről, különböző adatokat lehessen kényelmesen az egész programban használni. Ezek nagyrészt ID-k, de több esetben is lehet teljes lista, név vagy leírás.

Ehhez a React useContext rendszerét használtuk, hogy további Context-eket és Provider-eket

készíthessünk. A Provider-ekbe vannak csomagolva az útvonalak, vagyis a komponensek, melyeket használ a weboldal. Mivel ez a fő része a webalkalmazásnak, így ebbe minden komponens beletartozik.

Maguk a fájlok tartalmazzák a Contexteket.

Ezek itt vannak létrehozva. Később másik fájlokban ezek a useContext hook-kal

vannak használva. Az itt példaként felhozott 'setter' fogja a

RaceContext-en belül átállítani azt a useState-es változót, amit máshol is lehet használni. Ezek a Provider függvényben

vannak létrehozva, és ezekbe vannak csomagolva az oldalak, amikben a „children” lesz maga az oldal.

```

BgNameProvider>
BgDescProvider>
BgSkillsProvider>
BgToolProvider>
FinalSpellsProvider>
StatsProvider>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/creator-options" element={<CreationOptionsPage />} />
    <Route path="/character-creator" element={<CreatorPage />} />
    <Route path="/finalize" element={<CollectedDataTest />} />
    <Route path="/sign-in" element={<SignInPage />} />
    <Route path="/sign-up" element={<SignUpPage />} />
    <Route path="/profile" element={<ProfilePage />} />
  </Routes>
</StatsProvider>
FinalSpellsProvider>
BgToolProvider>
BgSkillsProvider>
BgDescProvider>
BgNameProvider>

```

14. ábra - Frontend Route-ok

```

//Race Id
export function RaceIdProvider({ children }) {
  const [selectedRaceId, setSelectedRaceId] = useState(0);

  return (
    <RaceIdContext.Provider value={{ selectedRaceId, setSelectedRaceId }}>
      {children}
    </RaceIdContext.Provider>
  );
}

//Subrace Id
export function SubraceIdProvider({ children }) {
  const [selectedSubraceId, setSelectedSubraceId] = useState(0);

  return (
    <SubraceIdContext.Provider value={{ selectedSubraceId, setSelectedSubraceId }}>
      {children}
    </SubraceIdContext.Provider>
  );
}

```

15. ábra - Provider kód részlet

```

const { setSelectedRaceId } = useContext(RaceIdContext);
const { setSelectedSubraceId } = useContext(SubraceIdContext);

```

16. ábra - Változó deklarációk

```

export const RaceIdContext = createContext();
export const SubraceIdContext = createContext();

```

17. ábra - Race és Subrace exportok

DiceRoller

A DiceRoller, avagy a funkció, amivel kockákat lehet dobni a DnD egyik

```
import React, { useEffect, useState } from "react";
import "./RollerStyles.css"; // Import CSS for styling
```

18. ábra - Dice Roller importok

alappillére. A játékban minden támadás, védelem, interakció, sőt, akár a sors is egy dobáson múlhat. Ezért a webalkalmazásunkba elengedhetetlennek tartottuk implementálni ezt a funkciót.

A stat-okat is meg lehet adni kockák dobása által. Erre a legtöbbet használt verzió az ún. „roll four drop the lowest”. Elég beszélő neve egyértelművé teszi, hogy mit kell csinálni: dobni négy darab d6-ot, és a legkisebb dobást nem beleszámítani (több ugyanakkora legkisebb dobás esetén is csak egy nincs számítva).

```
function Roll()
{
  var counter = 0;
  var rolls = "| ";

  var Amount = document.getElementById("AmountInput").value;
  var Size = document.getElementById("SizeInput").value;

  for (var i = 0; i < Amount; i++)
  {
    var temp = (Math.floor(Math.random() * Size) + 1)
    counter += temp;
    rolls += temp.toString() + " | "
    console.log(counter)
  }

  setRolledValue(counter);
  setRolls(rolls);
}
```

19. ábra - Dice Roller Roll függvénye

A DiceRoller-nek saját CSS fájlja van, mivel nem használhatjuk ugyanazt a formázást, amit a kártyákhoz, mert eltérő esztétikát készítettünk neki.

A rolledValue változó a teljes összeget tárolja, ami hozzáadja a dobott mennyiségszer a dobott értékeket. A rolls változó pedig eltárolja a rollokat külön.

```
return (
  <div className={"dice-roller ${isVisible ? "show" : "hide"}"}>
    <h3>{rolledValue}</h3>
    <p>{rolls}</p>

    {/* Number Inputs */}
    <div className="dice-inputs">
      <label>
        Dice Amount:
        <input type="number" min="1" defaultValue="1" id="AmountInput"/>
      </label>
      <label>
        Dice Size:
        <input type="number" min="2" step="2" defaultValue="6" id="SizeInput"/>
      </label>
    </div>

    {/* Hexagon Roll Button */}
    <div className="hexagon-btn-container">
      <button className="hexagon-btn" onClick={() => {Roll()}}>Roll</button>
    </div>
  </div>
);
```

20. ábra - Dice Roller return kódja

Maga a DiceRoller dizájnya egy dobótálcát imitál. A D&D játékosok ezeket előszeretettel használják nemcsak dobásokra, hanem kockák eltárolására is.

```
<li className="nav-item">
  <button
    className="btn btn-warning"
    onClick={() => setShowDiceRoller((prev) => !prev)}
  >
    Dice Roller
  </button>

  {/* Dice Roller Component (conditionally rendered) */}
  <DiceRoller isVisible={showDiceRoller} />
</li>
```

21. ábra - Dice Roller dizájn kód részlet

A Roll gomb pedig egy d20 oldalnézetére utal. A dobás összege és a dobások külön is esztétikusan meg vannak jelenítve, hogy a felhasználói igényeit dobástól függetlenül kielégítse.



22. ábra - Dice Roller dizájnja

Gyakran használt algoritmusok

diceToInteger

Ez az algoritmus viszonylag kevés fájlban van használva, legalább is a többi ilyenhez képest, viszont ettől függetlenül is fontos, mivel segít rövidíteni és érthetőbbé tenni a kódot. A függvény szöveges értéket vesz be meghíváskor, amely az általunk használt esetekben a 'd', majd 'az adott

dobókocka oldalszáma' formátumban van. A függvény maga ezen adat alapján visszaadja az oldalszámot integerként. Ez megkönnyíti a HitDie és DamageDie adatok feldolgozását.

```
function diceToInteger(die)
{
    die = die.substring(1, die.length);

    die = parseInt(die);

    return die;
}

export default diceToInteger;
```

23. ábra - diceToInteger függvény kódja

fetchEverything

Ez az algoritmus talán a leghasználtabb az összes közül a kódunkban. A fetchEverything volt a legelső algoritmus, amelyet megírtunk, és az egyik, ha nem leghasznosabb. Nagyon egyszerű a felépítése: egy szöveges adatot vár, amely, ha null, egy üres szöveges

adattá teszi. Ezt a szöveges adatot (pl.: 'FeaturesToFeaturesConnections/originclassid/3') az algoritmus hozzáfűzi az API elérési útjához('https://localhost:7188/api/'), és ezzel a linkkel indít egy GET kérést az API felé. Mivel async függvény, megvárja a kapott választ, eltárolja egy változóban, majd visszaadja.

```
async function fetchEverything(choice)
{
    if (choice == null)
    {
        choice = "";
    }

    const response = await fetch('https://localhost:7188/api/'+choice);
    var data = await response.json();

    return data;
}

export default fetchEverything;
```

24. ábra - fetchEverything aszinkron függvény kódja

modCalc

Ezen algoritmus egyike három 'calc' kiszámoló algoritmusnak, amit néhány helyen felhasználunk, ezért tisztábbnak tartottuk azt, hogy ezeket külső .js fájlokból hívjuk meg a lokális létrehozás helyett. A modCalc specifikusan egy integert vár, majd ez alapján kiszámolja a stat-hoz tartozó modifier-t. A kiszámításnál tízet kivon a stat-ból, majd elosztja kettővel, és a Math.floor() beépített algoritmus segítségével a legközelebbi, nála kisebb

egészre kerekíti, és ezt az értéket adja vissza. Ezt az értéket használja a DnD a számos tevékenység sikerességének a vizsgálatához (pl.: erő-próba: dobsz egy 20 oldalú kockát és a dobott értékhez hozzá adod a karaktered Strength stat modosítóját, valamint további bónuszokat is lehet szerezni különböző effektekből amelyeket nagyon részletekbe menően taglalnak a könyvek).

```
function modCalc(stat)
{
    var Mod = (stat - 10) / 2;
    Mod = Math.floor(Mod)
    return Mod;
}

export default modCalc;
```

25. ábra - modCalc függvény kódja

profCalc

A profCalc is egy kiszámító függvény, amely egy adott karakter 'szakértelmi bónuszát', avagy profiency bonus-t számítja ki a karakter szintje alapján. Integer típusú adatot vár, és ezzel számol. Alapból a vissza adandó értéket (PB) kettőre állítja, amely a legkisebb lehetséges profiency bonus. Ezután a karakter szintjéből (a várt adat) kivon egyet, majd elosztja néggyel, és az így kapott értéket hozzáadja a vissza

```
function profCalc(Level)
{
    let PB = 2;
    PB += (Level - 1) / 4
    PB = Math.floor(PB)

    return PB;
}

export default profCalc;
```

26. ábra - profCalc függvény kódja

adandó értékhez, és ezt vissza is adja. A számítási módszer nem hivatalos, mivel a legtöbb DnD-vel kapcsolatos könyvben és programban ezt a játékos számára elérhető Class táblázatokban adják meg, de azzal, hogy ezt a felelősséget is levettük a játékos válláról, is arra törekedtünk, hogy egyszerűbbé tegyük a karakter készítés folyamatát. Ezt azt értéket számos tevékenységhez, attribútumhoz vagy képességhez hozzá engedi adni a játék, például fegyverrel való támadáshoz amennyiben profiency vagy a fegyverrel, azaz tapasztaltan tudsz bánni vele akkor hozzá adhadod ezt az értéket a 20 oldalú kockán való dobás értékéhez az

adott attribútum módosítója mellet (különböző fegyvereket más képességgel kell használni, főként Strength, vagy Dexterity).

spellLevelCalc

Ezen algoritmus egy nagyon specifikus adatot számít ki, de ennek ellenére fontos, ugyanazon okok miatt, mint az előzőekben taglalt profCalc. A játékosnak a legtöbb esetben ki kellene számolnia, vagy egy adott Class táblázatában meg kellene keresnie, melyik a karaktere által használható legmagasabb szintű varázslat.

```
function spellLevelCalc(spellCaster, halfCaster, characterLevel)
{
    var spellLevel = 1;
    if(spellCaster & !halfCaster & characterLevel <= 17)
    {
        spellLevel = (Math.round(characterLevel/2));
    }
    else if(!spellCaster & halfCaster & characterLevel <= 17)
    {
        spellLevel = (Math.round(((characterLevel-1)/4)));
    }
    else
    {
        spellLevel = -1;
    }

    return spellLevel;
}

export default spellLevelCalc
```

27. ábra - spellLevelCalc függvény kódja

De ebben az esetben ez a függvény a szint alapján kiszámítja, hogy a felhasználó a varázslatok kiválasztásánál csak a karaktere számára elérhető varázslatokat lássa. Maga az algoritmus vár három adatot: két olyan integert, amelyek 0 vagy 1 értékűek lehetnek (spellCaster és halfCaster), illetve a karakter szintjét(characterLevel). Ezek után le ellenőrzi, a három lehetőséget az előbbi két változó értékét illetően, mivel nincs olyan Class amelynek mind a spellCaster és halfCaster tulajdonsága egyes értékkel rendelkezne. Abban az esetben, ha a spellCaster egyes értékű, a legnagyobb elérhető varázslat szintjét úgy számítja ki, hogy a karakter szintjét elosztja kettővel, majd kerekíti a legközelebbi egész számra. Másik esetben, ha csak a halfCaster egyes értékű, akkor hasonlóan számítja, mint a profCalc: a karakter szintjéből kivon egyet, majd elosztja négyvel és a legközelebbi egész számra kerekíti. Abban az elvileg nem létező esetben, ha mind a kettő nulla értékű, mínusz eggyé teszi a vissza adandó spellLevel változót, mivel ez azt jelzi, hogy a karakter nem rendelkezik varázslatokkal.

fetchIntoDoubleList

Ezt az algoritmust több helyen használjuk opciók felvételére egy kiíratáshoz használt listába. A képen látható esetben a 'RaceCard' komponensben van felhasználva, hogy az API válaszában forEach-el végig menjünk, és egy temp2 listába felvegyük az adott

```
/*Fetching all the races at startup */
useEffect(()=>{
  async function fetchIntoDoubleList()
  {
    const raceOptionsJSON = await fetchEverything("Races");

    var temp = [];
    var temp2 = [];
    raceOptionsJSON.forEach(race => {
      temp2.push(race.name);
      temp2.push(race.id);

      temp.push(temp2);
      temp2 = [];
    });
    setRaceOptionNames(temp);
  }
  fetchIntoDoubleList();
}, []);
```

28. ábra - fetch beágyazott listába

Race nevét és Id-ját, majd az adott iteráció végeztével ezt a temp2 listát felvegyük a temp lista következő elemeként. Ez a módszer abban segít, hogy csak a releváns információkon menjen végig a program a tényleges rendereléskor, ezzel is növelve a hatékonyságot és csökkentve az egyidejű erőforrásigényt.

spellSelect

Ez az algoritmus a SpellCard-ban van felhasználva, abban az esetben, ha a felhasználó kiválaszt egy adott varázslatot az elérhető varázslatok listájából. Az algoritmus maga felépítésében egyszerű, és csak két adatot vár: két integer-t, a varázslat szintjét és Id-át. Először megnézi, hogy a felhasználó által már kiválasztott varázslatok (a 0. szintűeket külön kezelve,

```
function spellSelect(id, level) {
  if (chosenSpellIds.includes(id)) return;

  const isCantrip = level === 0;

  if (isCantrip) {
    if (chosenCantripsNumber >= chosenCantripsLimit) return;
    setChosenCantripsNumber(prev => prev + 1);
  }
  else {
    if (chosenSpellsNumber == chosenSpellsLimit) return;
    else setChosenSpellsNumber(prev => prev + 1);
  }

  setChosenSpellIds(prev => [...prev, id]);
  setFinalSpells(prev => [...prev, id]);
}
```

29. ábra - spellSelect függvény részlet

mivel azoknak van egy sajátos nevük) száma 0-s, illetve annál nagyobb szintű varázslatok esetében meghaladja-e a maximum kiválasztható varázslatok számát, vagyis a felhasználó választhat-e még ki varázslatokat az adott kategóriában (ezért várja a varázslat szintjét, mint adatot). Ezek után egy temparray lokális listába lemásolja a felhasználó által kiválasztott varázslatok Id-ait tartalmazó listát, illetve létrehoz egy lokális integer típusú változót, amely

felveszi a kiválasztott 0. szintű varázslatok számát (továbbiakban Cantrip-k). ezután, ha a kiválasztott varázslatok között nem található a kiválasztandó varázslat Id-a, akkor hozzáadja ahhoz a listához, illetve növeli a kiválasztott Cantrip-k (ha a kiválasztandó varázslat szintje 0), vagy a kiválasztott varázslatok számát.

spellDeselect

Ezen algoritmus mondhatni, hogy a spellSelect inverze. Az algoritmust akkor használjuk, amikor a felhasználó el szeretné távolítani valamelyik varázslatot

```
function spellDeselect(id, level) {
  const isCantrip = level === 0;
  setChosenSpellIds(prev => prev.filter(spellId => spellId !== id));
  setFinalSpells(prev => prev.filter(spellId => spellId !== id));
  if (isCantrip) {
    setChosenCantripsNumber(prev => Math.max(0, prev - 1));
  }
  else {
    setChosenSpellsNumber(prev => Math.max(0, prev - 1));
  }
}
```

30. ábra - spellDeselect függvény részlet

az általuk választott varázslatok listájából. Működése ugyanazon logikán alapul, mint a spellSelect, csak míg az hozzáad a varázslatokat tartalmazó listához, illetve a kiválasztott Cantrip-k számához, a spellDeselect elvesz ezekből. Illetve ezen algoritmus esetében nem szükséges ellenőrizni, hogy a maximum varázslatok számához képest kisebb vagy egyenlő a kiválasztott varázslatok száma, Cantrip-k és magasabb szintű varázslatok esetében sem.

Karakter mentés

Az elmentett karakterek különleges módon vannak elmentve, hogy a lekérdezés egyszerűbb legyen. Minden kész sheet le van generálva a kreálás után, az adatokkal, amiket a felhasználó kiválasztott. Ezt követően megkapja a sheetet, és ad a karakterének egy nevet,

```
<div class="col-md-2">
  <label class="form-label fw-bold">Class</label>
  <div id="characterClass">${classDataResponse.name}</div>
</div>

<div class="col-md-2">
  <label class="form-label fw-bold">Level</label>
  <div id="characterLevel">${FinalCharacterLevel}</div>
</div>

<div class="col-md-2">
  <label class="form-label fw-bold">Race</label>
  <div id="characterRace">${raceDataResponse.name}</div>
</div>

<div class="col-md-2">
  <label class="form-label fw-bold">Background</label>
  <div id="characterRace">${BgName}</div>
</div>
```

31. ábra - Character Sheet dizájn kódja

beír információkat, amiket rendszerrel nem lehet kiválasztani, például olyan képességek, amik úgy nevezett Advantage-t adnak, azaz a játékos dob két kockát, és a nagyobb dobás eredményét használja, és elmenti a karakterét. Ez teljes mértékben HTML formátumban van, így bármikor tudja a felhasználó változtatni az adatait a későbbiekben.

sendSignIn

A sendSignIn algoritmus a SignInPage komponensen belül található meg és gyakorlatilag a teljes komponens leglényegesebb része, mivel ez használja fel a komponensben a felhasználó által megadott adatokat. Először egy-egy változóba felveszi az API elérési útját, majd felveszi JSON formátumban a felhasználó által megadott adatokat. Ezek után a program (ha lehetséges, try és catch segítségével) egy új változóba létrehozza a response-t, belefűzve az előzőekben felvett adatokat, illetve egy header-t, illetve megadja a POST method-t is. Majd ezt ágyazza egy fetch utasításba és elküldi az API-nak. Ha a visszakapott válasz nem egy 200-as OK response, akkor a felhasználó számára kidob egy alert-t, amely leírja, hogy a jelszó vagy a megadott e-mail cím helytelen, illetve kiírja a pontos hibát

a konzolba, hibakezelés és fejlesztői diagnosztika céljából. Abban az esetben, ha a response tartalmaz egy token-t (amelyet minden sikeres bejelentkezésnél tartalmaznia kell), ezt a token-t elteszi egy változóba (tesztelés és későbbi fejlesztés céljából), és eltárolja azt a localStorage-ben. Emellett felveszi a UserId változóba a bejelentkezett felhasználó azonosítóját, hogy azt a Context tovább küldhesse a főoldalra. Abban az esetben, ha valami ok miatt nem kapna token-t a response-ban, kiír egy hibát a konzolra. Illetve, ha az eredetileg küldött fetch nem futna le, akkor ki írja azt a hibát szintén a konzolra.

```
const sendSignIn = async () =>
{
  const url = "https://localhost:7188/api/Auth/login";
  const data =
  {
    "email": temp,
    "password": temp2
  };

  try {
    const response = await fetch(url,
    {
      method: "POST",
      headers:
      {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(data)
    });

    if (!response.ok)
    {
      window.alert("Invalid e-mail address or password.");
      navigate("/sign-in");
      throw new Error("Network response was not ok");
    }
  }
```

32. ábra - sendSignIn Post Method kódja

```
if (responseData.token)
{
  localStorage.setItem("authToken", responseData.token);
  setToken(responseData.token);
  setUserId(responseData.userId);
  navigate("/"); // Navigate to home after successful sign-in
} else
{
  window.alert("Failed to retrieve token.");
}

}

catch (error)
{
  console.error("Error:", error);
}
```

33. ábra - Token és UserId localStorage mentése

Tesztelési dokumentáció

Teszteset #1 – Helyes Felhasználó létrehozása

Teszteset azonosító: 001

Teszt típus: Funkcionális, Fehér doboz

Tesztelt funkció: Felhasználó létrehozása

Lépések:

1. Belépés a DnD Above weboldalra.
2. A weblapon Log in to existing account gombra kattintás
3. Email, és jelszó mező helyes kitöltése
4. 'Sign In' gombra kattintás

Elvárt eredmény: A felhasználó visszakerül a bejelentkezett kezdőképernyőre.

Valós eredmény: A felhasználó visszakerül a bejelentkezett kezdőképernyőre.

Teszteset #2 – Hibás Felhasználó létrehozása

Teszteset azonosító: 002

Teszt típus: Funkcionális, Fekete doboz, Negatív

Tesztelt funkció: Felhasználó létrehozása

Lépések:

1. Belépés a DnD Above weboldalra.
2. A weblapon Log in to existing account gombra kattintás
3. A név, email és jelszó mező kitöltése, email-be nem szerepel '@' karakter
4. 'Sign In' gombra kattintás

Elvárt eredmény: A használó marad a felhasználó létrehozásához használt felületre egy hiba üzenettel, hogy az email hibás, kötelező a @ karakter.

Valós eredmény: A használó marad a felhasználó létrehozásához használt felületre egy hiba üzenettel, hogy az email hibás, kötelező a @ karakter.

Teszteteset #3 – Helyes karakterkészítés

Teszteteset azonosító: 003

Teszt típus: Funkcionális, Fekete doboz

Tesztelt funkció: Karakter létrehozása

Előfeltétel: Bejelentkezett felhasználó

Lépések:

1. 'Create a Character' gomb kiválasztása.
2. Stat elosztási rendszer kiválasztása
3. Stat-ok megadása
4. Kívánt karakter szintjének megadása
5. Race: Human
6. Subrace: Variant Human
7. Background megadása
8. Class: Fighter
9. Subfeature-ok kiválasztása, vannak feature-ok melyek vonzanak magukkal választási opciót.
10. Subclass kiválasztása
11. Equipment-ek, felszerelések kiválasztása a megadott gold mennyiségből.
12. Spell-ek kiválasztása, ha megjelenik az opció, csak megadott Class-ok, fajok, Subclass-ek esetén jelenik meg.
13. Karakter mentése

Elvárt eredmény: A karakter megjelenik a „Saved Characters” listában.

Valós eredmény: A karakter megjelenik a „Saved Characters” listában.

Teszteteset #4 – Hibás karakterkészítés

Teszteteset azonosító: 004

Tesztstípus: Funkcionális, Fekete doboz, Negatív

Tesztelt funkció: Karakter létrehozása

Előfeltétel: Bejelentkezett felhasználó

Lépések:

1. „Create a Character” gomb kiválasztása.
2. Stat elosztási rendszer kiválasztása
3. Stat-ok megadása
4. Race: Human
5. Subrace Variant Human
6. Background megadása
7. Class: Sorcerer
8. Szint megadása
9. Subfeature-ok kiválasztása, vannak feature-ok melyek nem vonzanak magukkal választási opciót.
10. 'Finalize Selection' gomb megnyomása
11. Subclass kiválasztása
12. Equipment kiválasztása a megadott gold mennyiségből.
13. Spell-ek kiválasztása, ha megjelenik az opció, csak megadott Class-ok, fajok, Subclass-ek esetén jelenik meg.
14. 'Save Character' gomb megnyomása
15. ASI-k elosztása
16. 'Finalize Character' gomb megnyomása

Elvárt eredmény: Nem változik a felület, 'You need to select a race and a class' üzenet jelenik meg.

Valós eredmény: Nem változik a felület, 'You need to select a race and a class' üzenet jelenik meg.

Teszteset #5 – Elkészített karakter mentésének kihagyása

Teszteset azonosító: 005

Teszt típus: Funkcionális, Fekete doboz, Negatív

Tesztelt funkció: Karakter létrehozása

Előfeltétel: Bejelentkezett felhasználó

Lépések:

1. „Create a Character” gomb kiválasztása.
2. Stat elosztási rendszer kiválasztása
3. Stat-ok megadása
4. Race: Human
5. SubRace Variant Human
6. Background megadása
7. Class: Sorcerer
8. Szint megadása
9. Subfeature-ok kiválasztása, vannak feature-ok melyek nem vonzanak magukkal választási opciót.
10. 'Finalize Selection' gomb megnyomása
11. Subclass kiválasztása
12. Equipment kiválasztása a megadott gold mennyiségből.
13. Spell-ek kiválasztása, ha megjelenik az opció, csak megadott Class-ok, fajok, Subclass-ek esetén jelenik meg.
14. Oldal bezárása

Elvárt eredmény: Nincs mentés a Character-ről.

Valós eredmény: Nincs mentés a Character-ről.

IV. Felhasználói dokumentáció

A program általános specifikációja, célja

A DnD Above egy asztali webalkalmazás, amely a Dungeons & Dragons 5. kiadásához (D&D 5e) kínál digitális támogatást. Célja, hogy a hagyományos, papíralapú karakterlapokat digitálisan, könnyen kezelhető és bármikor elérhető formában biztosítsa a játékosok számára.

A program fő funkciója a karakterlapok (character sheet-ek) létrehozása, szerkesztése és tárolása egy jól strukturált, modern felhasználói felületen keresztül. A felület lépésről lépésre vezeti végig a játékost a karakterkészítés folyamatán, a faj és osztály kiválasztásától a varázslatok és felszerelés megadásáig.

A rendszer automatikusan kiszámítja az olyan játéktechnikai értékeket, mint az életerő (HP), támadási módosítók, mentődobások és képességpróbák. Emellett a Dice Roller funkció segítségével a játékosok egyszerűen dobhatnak kockával a programon belül, így nincs szükség külön fizikai kockákra.

A program célközönsége elsősorban új játékosok, akik most ismerkednek a DnD világgal, de azok a haladó felhasználók is hasznát veszik, akik szeretnék digitálisan megszerezni és könnyen elérhetővé tenni karaktereiket.

A DnD Above offline módban is működőképes, a DndAbove.exe fájl elindításával a backend automatikusan elindítja a frontend felületet a böngészőben. Így a program külön telepítést nem igényel, és teljes funkcionalitását internetkapcsolat nélkül is biztosítani tudja.

Fő célkitűzésünk egy egyszerűen kezelhető, vizuálisan áttekinthető és gyors rendszert létrehozni, amely támogatja a karakteralkotás komplex lépéseit, valamint segíti a karakterlap hosszú távú tárolását, szerkesztését és kezelését.

Rendszerkövetelmények

Hardverkövetelmények:

Minimális rendszerkövetelmények (felhasználóknak):

- **Processzor:** Intel Core i3-4130 / AMD Ryzen 3 1200
- **Memória (RAM):** 4 GB
- **Tárhely:** 300 MB szabad lemezterület
- **Kijelző:** Minimum 1280x720 (HD felbontás)
- **Internetkapcsolat:** Stabil, legalább 5 Mbps letöltési sebesség
- **Böngésző:** Bármely korszerű HTML5-támogatású böngésző (lásd lejjebb)

Ajánlott rendszerkövetelmények (fejlesztőknek és haladó felhasználóknak):

- **Processzor:** Intel Core i5 (8. generáció) vagy AMD Ryzen 5 és újabb
- **Memória (RAM):** 8 GB vagy több
- **Tárhely:** Legalább 1 GB szabad lemezterület a forráskód, build fájlok és adatbázis kezeléséhez
- **Kijelző:** Full HD (1920x1080) vagy nagyobb felbontás
- **Internetkapcsolat:** 10 Mbps vagy gyorsabb

Szoftverkövetelmények:

Felhasználói oldalon:

- **Operációs rendszerek:**
 - Windows 10 vagy újabb
 - macOS 10.15 (Catalina) vagy újabb
- **Böngészők (frissített verziók):**
 - Google Chrome
 - Mozilla Firefox
 - Microsoft Edge
 - Safari (macOS-en)
- **Egyéb követelmények:**
 - JavaScript engedélyezett állapotban a böngészőben
 - HTML5 és CSS3 támogatás

Fejlesztői környezethez:

- **Fejlesztői eszközök:**
 - Microsoft Visual Studio 2022 (C# és .NET támogatással)
 - Visual Studio Code

- Node.js és npm (a React frontendhez)
- Vite keretrendszer
- **Futtatókörnyezet:**
 - .NET 6 vagy újabb (Backend alkalmazáshoz)
 - SQLite adatbázismotor (lokálisan)
 - DB Browser for SQLite (adminisztrációhoz opcionális)

A program telepítése

A DnD Above program nem igényel klasszikus telepítést, azonban a működéséhez szükséges a DndAbove.exe fájl elindítása, amely elindítja a háttérben futó szerveret (backend), majd automatikusan megnyitja az alkalmazás felületét az alapértelmezett böngészőben. Az alábbi lépések részletesen bemutatják a program indítását.

1. Fájlok előkészítése (1)

- a. Nyisd meg a DnD Above GitHub oldalát a böngésződben.
- b. Navigálj a Releases szekcióhoz, és töltsd le a legfrissebb verziót tartalmazó .zip fájlt.
- c. Csomagold ki a letöltött .zip fájlt egy tetszőleges mappába a számítógépeden (pl. C:\DnDAbove).
- d. Ellenőrizd, hogy a mappában megtalálható legyen a következő fájl:
 - DndAbove.exe

2. Fájlok előkészítése (2)

- a. Kapott csomagban a Github_link.txt nevű fájlhoz navigáljon
- b. Nyissa meg majd ctrl gomb nyomva tartása közben kattintson a linkre Vagy másolja ki a linket a preferált böngészőjében
- c. Folytassa a 'Csomagold ki a letöltött .zip fájlt egy tetszőleges mappába a számítógépeden (pl. C:\DnDAbove).' ponttól az első fájl előkészítés menü pontból

2. Tűzfal- vagy vírusirtó figyelmeztetés

- a. Előfordulhat, hogy a rendszer rákérdez, hogy engedélyezi-e a DndAbove.exe futtatását.
- b. Ilyen esetben kattints az **„Engedélyezés”** vagy **„Futtatás mindenképp”** lehetőségre.

3. A program indítása

- a. Kattintson duplán a DndAbove.exe fájlra.
- b. A fájl elindítja a háttérben futó webszerveret.

- c. Majd megtudja nyitni az ön által preferált böngészőjében az oldalt a localhost:5173 címen
- 4. A program bezárása
 - a. A program bezárásához egyszerűen zárja be a megnyílt böngészőlapot.
 - b. Valamint a háttérben futó cmd ablakot is zárja be és azzal leállítja a programot is
- 5. Fontos megjegyzés
 - a. A program nem igényel internetkapcsolatot, ha minden fájl helyben van.
 - b. A DndAbove.exe elindítása nélkül a böngészőből nem lehet megnyitni az alkalmazást.

Ez a telepítési és indítási folyamat lehetővé teszi, hogy a felhasználó néhány kattintással hozzáférjen az alkalmazáshoz, külön konfigurálás nélkül.

A program használatának a részletes leírása

A program használatának megkezdése

Weboldal megnyitása

DnDAbove.exe fájl elindítása, a weboldal megnyitása localhost:5173 url-n

Regisztráció, ha még nincs fiókja

1. Kattintson a '**Create a new account**' (Regisztráció) gombra.
2. Töltse ki az alábbi mezőket:
 - **Username**
 - **Email address**
 - **Password**
3. Töltse le és olvassa el az Eula dokumentumot.
4. Eula elfogadása.
5. Kattintson a '**Sign up**' (Fiók létrehozása) gombra.

Bejelentkezés

1. Kattintson a **Log in to existing account** (Bejelentkezés) gombra.
2. Írja be a regisztrált email címet és jelszót.
3. Kattintson a '**Sign In**' (Belépés) gombra.

Karakterkészítés lépésről lépésre

Karakter létrehozása

1. Ha be van jelentkezve
 - 1.1. Nyomja meg a 'Create New Character' gombot
2. Ha nincs bejelentkezve és nincsen regisztrálva
 - 2.1. Térjen át a Regisztráció, ha még nincs fiókja ponttal, majd térjen vissza a karakter készítés elejéhez.
3. Ha nincs bejelentkezve de van fiókja
 - 3.1. Térjen át a Bejelentkezés ponttal, majd térjen vissza a karakter készítés elejéhez.
4. Kattintson a 'Create New Character' gombra:
 - 4.1. Extra ASI lehetőség kiválasztása, ha DM, azaz játékmester azt engedélyezi, szóban.
 - 4.2. A karakter stat elosztási rendszerének kiválasztása
 - 4.2.1. Ha 'Point Buy' rendszer választott a felhasználó:
 - 4.2.1.1. Osszon el 27 pontot Strength, Dexterity, Constitution, Intelligence, Wisdom, Charisma stat-ok között, minden stat 8-rol kezd

és egyik sem mehet 15 fölé, és 13 fölött kettőbe kerül az eggyel való növelés.

4.2.2. Ha 'Heroic Point Buy' választott a felhasználó:

4.2.2.1. Osszon el 27 pontot Strength, Dexterity, Constitution, Intelligence, Wisdom, Charisma stat-ok között, minden stat 8-ról kezd és egyik sem mehet 18 fölé.

4.2.3. Ha 'Custom or Rolled'-t választott a felhasználó:

4.2.3.1. Dobjon ki a felhasználó 4d6 kockát magának vagy a beépített Dice Roller funkcióval.

4.2.3.2. A 3 legnagyobb értéket adja össze.

4.2.3.3. Ismételje meg minden stat-hoz.

4.3. Adja hozzá a +2 és a +1 pontot két különböző stat-hoz.

4.4. Válassza ki a kívánt fajt a megjelenő listából.

4.5. Ha megjelenik a Subrace opció

4.5.1. Válasszon Subrace-t vagyis alfajt a megjelenő listából

4.6. Adja meg a Background 2 Skill-t és egy tool-t melyekhez készséget szerez a karakter, adja meg a nevét és a leírását, azaz történetét.

4.7. Válassza ki a kívánt a Class-t

4.8. Válassza ki a kívánt szintet

4.9. Ha valamilyen feature választási opciókat hozz fel válasszon közülük, ilyen például a fighter fighting style feature-je

4.10. Válassza ki a kívánt Subclass-t

5. Ha vannak subfeature-jei a SubClass-nek vagy Class-nek, akkor kattintson a Finalize gombra, ha végzett a Subfeature-k kiválasztásával.

6. Equipment-ek, azaz felszerelések kiválasztása a három menü pontból ('Weapons', 'Armor & Shields', 'Other'). Ezen felszereléseket az elején látható 'Gold' mennyiségből vásárolja és el is tudja adni.

7. Amennyiben olyan Class-t választott ki, amely Caster vagy Halfcaster, megjelenik az opció, hogy válasszon Spell-eket.

8. 'Select Spell Level' gomb minden, a Class számára elérhető spell-t kiad azon a szinten, amelyet kiválasztott.

9. Amennyiben a választott Class Caster és nem Halfcaster, magasabb szintű Spell-eket és Cantrip-eket is választhat.

9.1. Kattintson a 'Save Character' gombra

- 9.2. Mivel minden 4. szinten kap ASI feature-t minden Class ezért ezután tudja kiválasztani mibe szeretné ezeket a további stat pontokat elosztani, valamint, ha kiválasztotta a level 1 Feat-t akkor +1 ASI.
- 9.3. Majd kattintson a 'Finalize' gombra
- 9.4. Ezzel megfog jeleni a végleges karakter, a weboldal magától generálja a karakter nevét, ez nem változtatható, generáció a faj nevéből, a Class nevéből és a szintből készít egy nevet.

Meglévő karakter megnyitása

1. Ha nincs bejelentkezve és nincsen regisztrálva
 - a. Nem lesz meglévő karakter, amit meglehet nyitni, térjen át a Regisztráció, ha még nincs fiókja részhez
2. Ha nincs bejelentkezve de van fiókja
 - a. Térjen át a Bejelentkezés pontra, utána térjen vissza a Meglévő karakter megnyitása pont elejéhez.
3. Ha be van jelentkezve
 - a. Kattintson a 'Saved Characters' gombra
 - b. Megfognak jeleni a már elkészített karakterek listája, ez lehet üres is ha még nem készített el karaktert.
 - c. Kattintson az 'Open' gombra a kívánt karakter mellett.

Egyéb funkciók leírása

Dice Roller

1. Navigáljon az oldal tetején található Dice Roller gombhoz majd kattintson rá
2. A gombnyomásra meg kell jelennie egy kisebb ablaknak a képernyő jobb alsó részén.
3. Válassza ki hány kockát szeretne dobni.
4. Válassza ki milyen méretű, avagy hány oldalú kockát vagy kockákat szeretne dobni.
5. Nyomja meg a 'Roll' (dobás) gombot a kis ablak alján.
6. Olvassa le a kis ablak tetején a számot, ez a dobás össz értéke.
7. Alatta pedig a dobások értéke külön látható.

V. Összegzés

Általános Összegzés

A projekt célja egy olyan webalkalmazás létrehozása volt, amely lehetővé teszi a Dungeons & Dragons 5. kiadás karakterlapjainak (character sheet-ek) egyszerű, áttekinthető és digitális elkészítését és tárolását. A célkitűzésünk az volt, hogy főként kezdő játékosokat támogassunk a karakteralkotás nehézségeinek leküzdésében, de olyan eszközt hoztunk létre, amely a haladó játékosok számára is kényelmes, gyors és modern megoldást jelent.

A megvalósított funkciók közé tartozik a karakterkészítő varázsló, az automatikus értékszámítások, a spell- és felszereléskezelés, valamint a Dice Roller eszköz. A program reszponzív, több eszközön használható, és lehetővé teszi a karakterek böngészőből való elérését.

A projekt sikeresen teljesítette az alapvető célokat:

- A rendszer stabilan fut, hibakezelése megbízható.
- A karakterkészítési folyamat logikus, felhasználóbarát.
- A dizájn átlátható, a kezelőfelület könnyen használható.

További fejlesztési lehetőségek

A fejlesztés során számos ötlet és igény merült fel, amelyeket a jövőben érdemes lenne megvalósítani:

- **Többnyelvű támogatás:** Az alkalmazás fordítása magyarra vagy más nyelvekre, hogy nemzetközi játékosok számára is elérhető legyen.
- **Importálási lehetőség:** Korábban máshol vagy papíron elkészített karakterlapok feltöltése (pl. PDF vagy JSON formátumban).
- **Exportálási lehetőség:** A karakterlap PDF-ként való exportálása, nyomtatási lehetőség biztosítása.
- **Kampánykezelő modul:** Több karakter összekapcsolása egy kampányba, a mesélők (Dungeon Masterek) támogatására.
- **Homebrew rendszer:** Egyedi tartalmak (saját varázslatok, tárgyak, Class-ok) létrehozása és adatbázisba illesztése.
- **Feat-ek implementálása:** Minden 4. szinten az ASI-k helyett választható extra képességek.

Nem megvalósított funkciók

Időkeret vagy technikai korlátok miatt az alábbi funkciók nem kerültek be az első verzióba, de előkészítettük a lehetőségét a későbbi fejlesztésre:

- Felhasználók közötti karaktermegosztás
- Karakterfejlődés követése szintezéskor (Level-up modul)

Összességében a DnD Above egy erős alapot nyújt egy modern, rugalmas és felhasználóbarát karakterkezelő rendszerhez, amely jelentősen megkönnyítheti a DnD játékosok életét, különösen a digitális játékterek korában.

VI. Ábrajegyzék

1. ÁBRA - FELHASZNÁLHOZ KAPCSOLÓDÓ TÁBLÁK	9
2. ÁBRA - RACES TÁBLA	10
3. ÁBRA - CLASSES TÁBLA	10
4. ÁBRA - SUBCLASSES TÁBLA.....	11
5. ÁBRA - EQUIPMENTS TÁBLA	11
6. ÁBRA - FEATURES TÁBLA	12
7. ÁBRA - SPELLS TÁBLA.....	13
8. ÁBRA - REGISTERDTO MODELL.....	14
9. ÁBRA - SAVEBASEDISPLAYDTO MODEL	14
10. ÁBRA - REFRESHTOKEN MODEL.....	14
11. ÁBRA - FEATURESTOFEATURESCONNECTION MODEL	14
12. ÁBRA - MAPPINGSAVES KÓD RÉSZLET	15
13. ÁBRA - MAPPINGUSERS KÓD RÉSZLET.....	15
14. ÁBRA - FRONTEND ROUTE-OK	16
15. ÁBRA - PROVIDER KÓD RÉSZLET	16
16. ÁBRA - VÁLTOZÓ DEKLARÁCIÓK	16
17. ÁBRA - RACE ÉS SUBRACE EXPORTOK	16
18. ÁBRA - DICE ROLLER IMPORTOK.....	17
19. ÁBRA - DICE ROLLER ROLL FÜGGVÉNYE	17
20. ÁBRA - DICE ROLLER RETURN KÓDJA.....	17
21. ÁBRA - DICE ROLLER DIZÁJN KÓD RÉSZLET	17
22. ÁBRA - DICE ROLLER DIZÁJNJA	18
23. ÁBRA - DICETOINTEGER FÜGGVÉNY KÓDJA	18
24. ÁBRA - FETCHEVERYTHING ASZINKRON FÜGGVÉNY KÓDJA.....	18
25. ÁBRA - MODCALC FÜGGVÉNY KÓDJA.....	19
26. ÁBRA - PROFCALC FÜGGVÉNY KÓDJA.....	19
27. ÁBRA - SPELLLEVELCALC FÜGGVÉNY KÓDJA.....	20
28. ÁBRA - FETCH BEÁGYAZOTT LISTÁBA	21
29. ÁBRA - SPELLSELECT FÜGGVÉNY RÉSZLET	21
30. ÁBRA - SPELLDESELECT FÜGGVÉNY RÉSZLET	22
31. ÁBRA - CHARACTER SHEET DIZÁJN KÓDJA.....	22
32. ÁBRA - SENDSIGNIN POST METHOD KÓDJA	23
33. ÁBRA - TOKEN ÉS USERID LOCALSTORAGE MENTÉSE	23

VII. Irodalomjegyzék, hivatkozásjegyzék

- <https://dnd5e.wikidot.com/>
- <https://dnd5e.wikidot.com/spells>
- <https://dnd5e.wikidot.com/#toc0>
- <https://dnd5e.wikidot.com/lineage#toc0>
- <https://dnd5e.wikidot.com/barbarian>
- <https://dnd5e.wikidot.com/bard>
- <https://dnd5e.wikidot.com/cleric>
- <https://dnd5e.wikidot.com/druid>
- <https://dnd5e.wikidot.com/fighter>
- <https://dnd5e.wikidot.com/monk>
- <https://dnd5e.wikidot.com/paladin>
- <https://dnd5e.wikidot.com/ranger>
- <https://dnd5e.wikidot.com/rouge>
- <https://dnd5e.wikidot.com/sorcerer>
- <https://dnd5e.wikidot.com/warlock>
- <https://dnd5e.wikidot.com/wizard>
- <https://roll20.net/compendium/dnd5e/Equipment#content>
- <https://stackoverflow.com>
- <https://www.w3schools.com/cs/index.php>
- <https://www.reddit.com/r/reactjs/?rdt=45330>
- <https://learn.microsoft.com/en-us/dotnet/csharp/>
- <https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>
- <https://learn.microsoft.com/en-us/dotnet/>
- <https://sqlite.org/docs.html>
- <https://chatgpt.com/>